



1. Начинаем не с запроса, а с структуры промпта

Сейчас я хочу показать вам, что хороший промпт для программирования обычно можно собирать из повторяющихся смысловых частей. Нам не нужно каждый раз придумывать запрос с нуля.

Я буду использовать простую структуру: сначала контекст, затем конкретная задача, после этого входные данные, ограничения, требования к результату и способ проверки.

Обратите внимание, я специально не начинаю с фразы вроде «Ты лучший программист на Python». Такая фраза практически ничего не сообщает о задаче. Мне гораздо важнее дать модели технический контекст.

Первый универсальный каркас будет выглядеть так:

```
Контекст:
[где используется код, какой стек, что уже существует]

Задача:
[что именно необходимо сделать]

Исходные данные:
[код, интерфейс, структура данных или требования]

Ограничения:
[что нельзя менять или использовать]

Ожидаемый результат:
[что модель должна вернуть]

Проверка:
[по каким условиям мы поймём, что результат корректен]
```

Сейчас мы будем несколько раз использовать этот каркас, но заполнять его по-разному в зависимости от задачи.

2. Почему нельзя начинать с «напиши функцию»

Возьмём самый простой запрос:

```
Напиши функцию расчёта скидки.
```

Формально модель может его выполнить. Но здесь отсутствует почти вся информация, необходимая для реального проекта.

Неизвестно, на каком языке писать функцию. Неизвестно, какие аргументы она получает. Неизвестно, что считать скидкой. Неясно, как обрабатывать ошибочные значения. Неясно, разрешено ли менять тип результата.

Поэтому я не оцениваю такой промпт как «плохой» из-за его длины. Проблема в другом: он оставляет слишком много решений на усмотрение модели.

Теперь я превращу тот же запрос в технический шаблон.

3. Шаблон промпта для генерации нового кода

Для генерации кода я использую такой шаблон:

Контекст:

Проект написан на [язык и версия].

Используется [framework / библиотека / архитектура].

Новый код будет использоваться для [назначение].

Задача:

Реализуй [конкретный компонент].

Входные данные:

[описание параметров, типов и структуры данных]

Требуемое поведение:

[что должна делать реализация]

Обработка ошибок:

[какие ошибочные ситуации необходимо учитывать]

Ограничения:

Не изменяй [существующий интерфейс / другие файлы].

Не используй [запрещённые библиотеки или подходы].

Не добавляй функциональность, которой нет в требованиях.

Ожидаемый результат:

Верни готовый рабочий код.

Сохрани указанную сигнатуру.

Добавь [тесты / docstring / обработку ошибок], если это требуется.

Критерии проверки:

[конкретные примеры ожидаемого поведения]

Теперь разберём, почему я написал именно так.

В **Контексте** я описываю среду. Если проект работает на Python 3.13 и FastAPI, модель не должна неожиданно предложить Flask или JavaScript.

В **Задаче** я использую конкретный глагол. Не «помоги с функцией», а «реализуй функцию».

В **Ограничениях** я заранее уменьшаю область изменений. Это особенно важно, когда AI работает внутри существующего проекта.

А в **Критериях проверки** я превращаю субъективный запрос в проверяемое требование.

4. Заполняем шаблон генерации кода

Теперь я покажу конкретный вариант.

Пусть нам нужна функция расчёта итоговой стоимости корзины.

Я отправляю AI такой промпт:

Контекст:

Проект интернет-магазина написан на Python 3.13.

Денежные значения в этом учебном примере представлены типом `Decimal`.

Задача:

Реализуй функцию `calculate_total(items, discount_percent)`.

Входные данные:

`items` – список словарей вида:

```
{
    "price": Decimal,
    "quantity": int
}
```

`discount_percent` – `Decimal` от 0 до 100.

Требуемое поведение:

Для каждой позиции умножь `price` на `quantity`.

Сложи стоимость всех позиций.

Примени процентную скидку ко всей корзине.

Верни итоговую сумму типа `Decimal`.

Обработка ошибок:

Если `quantity` меньше 0, выброси `ValueError`.

Если `price` меньше 0, выброси `ValueError`.

Если `discount_percent` меньше 0 или больше 100, выброси `ValueError`.

Пустая корзина должна возвращать `Decimal("0")`.

Ограничения:

Не используй сторонние библиотеки.

Не изменяй формат входных данных.

Не добавляй налоги, доставку и другие бизнес-правила, которых нет в требованиях.

Ожидаемый результат:

Верни готовую реализацию функции `calculate_total`.

После функции добавь unit-тесты на unittest.

Критерии проверки:

Корзина с товаром price=100, quantity=2 и скидкой 10 процентов должна вернуть 180.

Пустая корзина должна вернуть 0.

Некорректная скидка должна приводить к ValueError.

Теперь я обращаю внимание студентов на последнюю часть.

Я не просто написал «код должен работать». Я дал конкретные наблюдаемые примеры. Благодаря этому и модель, и разработчик одинаково понимают ожидаемое поведение.

5. Когда шаблон генерации становится слишком подробным

Теперь покажу другую ошибку. Иногда после изучения prompt engineering появляется желание описать вообще всё.

Например, можно написать модели несколько страниц о компании, истории проекта, составе команды и внутренней инфраструктуре, хотя наша функция только складывает стоимость товаров.

Такой контекст не помогает решению.

Я задаю себе простой вопрос: может ли эта информация изменить правильную реализацию текущей задачи?

Если ответ нет, я её не передаю.

Например, верс